

InfoGuard: Network-Level Misinformation Suppression

Azerin Salahova Dmitriy Kuramshin

Abstract

Misinformation propagating during emergencies outpaces manual moderation, and per-post content classification carries training-data bias, fails on out-of-distribution content, and delegates the editorial judgement of “what is misinformation” to a model whose decision boundary cannot be audited per post. Bayiz and Topcu (2022) introduced a network-level alternative: a per-step linear program over a class-pair dropout matrix that suppresses misinformation cascades without reading post content. We adopt this framework as the suppression layer of InfoGuard and contribute four extensions that turn it into a deployable system. (1) A bidirectional Graph Convolutional Network (BiGCN) classifier on cascade trees, integrated as a confidence-gated input filter to the model fitter, never as a moderation layer. (2) An argued polarisation-discovery configuration (Louvain resolution 0.5, 5% minimum-class-fraction merging floor) without which the conspiracy subgraph is not recovered as a separate class and the suppression signal disappears. (3) An emergency-deployment framing with a specified bias-audit module that pauses the LP when per-category dropout rates diverge by more than 15%. (4) A future-work direction for retrieval-augmented counter-content generation grounded in trusted sources. On the WICO Graph dataset, the fitted SBM yields $k = 4$ polarisation classes and a sharp asymmetry concentrated on a single class pair (b^- entries 1.00 and 0.93, b^+ at the Laplace floor). End-to-end cascade-following at $(\alpha, \lambda) = (1.5, 1.0)$ reverses the cascade-size ordering: mean cascade size 38.5 for misinformation vs 42.8 for verified content, against 59.8 vs 56.1 at control. Early-collapse probability rises from near zero to 0.14 (misinformation) and 0.05 (verified), a $2.8\times$ ratio against misinformation. BiGCN reaches binary accuracy 0.84 on Twitter15 at fold 0 of a 5-fold cross-validation. The bias audit is specified but not implemented.

Keywords: misinformation suppression; cascade dynamics; bidirectional graph convolutional networks; stochastic block models; network alterations.

1 Introduction

Misinformation that propagates during emergencies — public-health crises, natural disasters, wartime information environments — outpaces manual moderation. The standard automated counter-measure is per-post content classification, but classifier accuracy degrades on out-of-distribution content, training-data bias propagates directly into moderation decisions, and the editorial judgement of “what is misinformation” is delegated to a model whose decision boundary cannot be audited per post. None of these properties is acceptable for a deployment with civil-liberties implications.

A different line of work attacks the problem at the network level. Bayiz and Topcu (2022) introduced an optimisation-based framework for misinformation suppression on social-network cascades. A per-step network alteration minimises misinformation spread while preserving a guaranteed branching ratio for verified content; the alteration reads only the polarisation classes of sender and receiver, never the post text.

This removes the per-post decision and the bias surface that follows from it. We adopt this framework as our suppression layer and contribute four extensions that turn it into a deployable system.

(1) BiGCN classifier on cascade trees, integrated as a confidence-gated input filter to the model fitter. Predicted labels enter the fitter only when the classifier’s confidence exceeds a configured threshold; the classifier never enters the suppression loop and cannot block any post directly.

(2) Argued choices for polarisation discovery. Two clustering parameters — Louvain resolution and a minimum-class-fraction merging floor — determine whether the conspiracy subgraph is recovered as its own polarisation class and therefore whether the suppression layer has any pathway to act on. We make this dependency explicit and report the operating values that recover the signal.

(3) An emergency-deployment framing with a specified bias-audit module that pauses the suppression loop when per-category dropout rates diverge by more than 15%. The audit is specified, not imple-

mented.

(4) A future-work direction for retrieval-augmented counter-content generation grounded in trusted sources, turning the system from a defensive tool into an offensive one.

The fitted model splits users into $k = 4$ polarisation classes with sizes [38113, 54956, 8484, 12682] across 114,235 nodes. The structural asymmetry between the two fitted SBM matrices is concentrated in a single class pair, $C_3 \leftrightarrow C_4$, where b^- has entries 1.00 (clipped from a raw value above the unit cap) and 0.93 while the corresponding b^+ entries sit at the Laplace floor. On WICO, the mean cascade size $\mathbb{E}[R_\infty]$ at control is 56.1 for verified content and 59.8 for misinformation; at $(\alpha, \lambda) = (1.5, 1.0)$ both fall to 42.8 and 38.5 and the cascade-size ordering reverses. The early-collapse probability $\Pr[R_\infty < 5]$ rises from near zero at control to 0.05 for verified content and 0.14 for misinformation, a $2.8\times$ ratio against misinformation. BiGCN on Twitter15 reaches binary accuracy 0.84 and binary F1 0.76 at fold 0 of a 5-fold cross-validation; folds 1–4 are pending.

Three components are out of scope. The bias-audit module is specified but not implemented, and we report no audit numbers. Azerbaijani-language deployment is part of the use-case framing, not of the measured pipeline; all training and evaluation use English-language data. BiGCN is not evaluated on WICO because WICO Graph cascades do not include per-node tweet text, so the end-to-end LP cascade-following on WICO uses dataset folder labels rather than BiGCN labels.

2 Methods

2.1 Background and prior work

The mathematical core of the suppression layer is taken from Bayiz and Topcu (2022). We summarise the framework here without re-deriving it; the original work contains the full theoretical treatment.

Content propagation is modelled as a discrete-time Susceptible-Infected-Removed (SIR) process on a directed random graph whose per-edge transfer probabilities follow a Stochastic Block Model (SBM). The block structure is a partition of users into k polarisation classes; the SBM is parameterised by a $k \times k$ matrix of inter-class transfer probabilities. Two SBMs are fitted from labelled cascade data: b^+ for verified content, b^- for misinformation. The two matrices

share the same partition and differ only in their entries.

A graph alteration rewrites per-edge transfer probabilities. We restrict to dropout alterations parametrised by a $k \times k$ matrix $d^* \in [0, 1]^{k \times k}$. Each entry d_{uv}^* scales every transfer probability between class u and class v : $d_{uv}^* = 1$ leaves the network unchanged, $d_{uv}^* = 0$ blocks every transfer along that class pair.

At each cascade step the system solves a linear program for d^* that minimises expected misinformation spread under d^* and b^- , subject to a hard constraint that expected verified-content spread under d^* and b^+ stays at least α times the current cascade size. When the constraint cannot be met, a soft variant is used instead, with a weight $\lambda \geq 0$ trading verified preservation against misinformation suppression. α and λ are the two operating parameters of the suppression layer; their settings are chosen empirically and reported in §2.6. The closed-form feasibility check, the LP itself, the soft variant, and the linearisation that makes both tractable are stated in Bayiz and Topcu (2022, eqs. 22–24).

Everything that follows in this section — the BiGCN integration, the polarisation-discovery configuration, the cascade-following implementation choices, and the deployment framing — sits on top of this framework.

2.2 System architecture

Figure 1 shows the five blocks of INFOGUARD and their data flow. The pipeline runs in two distinct modes: an offline mode that fits the SBM matrices from labelled cascade data, and an online mode that runs at every step of a live cascade, calls the LP optimiser, and applies the resulting dropout matrix to the simulator.

The architecturally novel block is the BiGCN predictor and its handoff to the SBM fitter. The handoff is intentionally narrow: BiGCN labels enter the fitter only when classifier confidence exceeds a configured threshold, and the fitter is the only block they ever touch. The LP optimiser, the dropout matrix, and the cascade-following simulator never see classifier output. This is the property that lets us combine a BiGCN with binary accuracy of 0.84 — a level that would not be acceptable for direct moderation — with a suppression layer that must hold itself to a much stricter standard.

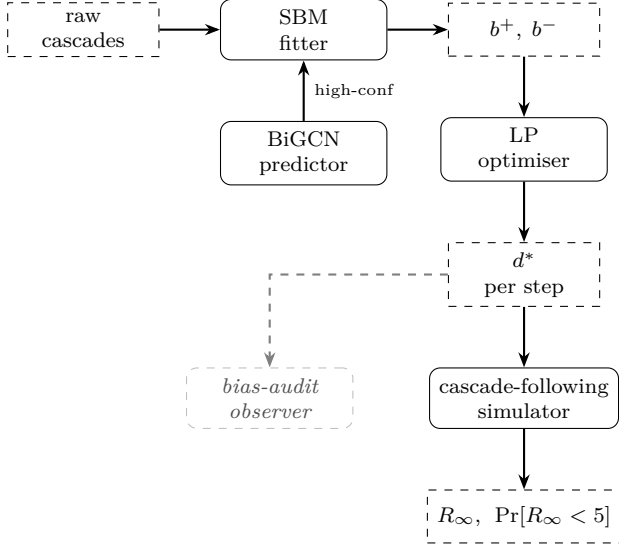


Figure 1: InfoGuard system architecture. Solid blocks are implemented; the bias-audit observer is specified but not implemented.

SBM fitter. Loads WICO Graph cascades, builds the union graph, runs Louvain modularity clustering with the configuration argued in §2.3, merges small classes, and estimates b^+ and b^- by per-class transfer counting. Saves the matrices to `data/processed/sbm_matrices/`.

LP optimiser. Solves the primary or soft LP from Bayiz and Topcu (2022) at every cascade step using `scipy.optimize.linprog` with the HiGHS backend. Returns the dropout matrix d^* together with diagnostic flags identifying which LP was solved.

BiGCN predictor. A bidirectional GCN trained on Twitter15 / Twitter16 propagation trees. Returns a binary label and a confidence score. Predictions whose confidence falls below 0.65 are discarded at the SBM-fitter boundary. The predictor never enters the LP loop.

Cascade-following simulator. Walks an observed WICO cascade in breadth-first order from its root. At each step it calls the LP optimiser, samples each forward edge as a Bernoulli trial with success probability d_{uv}^* , and accumulates the reachable set.

Bias-audit observer (specified, not implemented). Designed to read the dropout matrix at every step and accumulate per-content-category

dropout rates. If the largest pairwise rate gap exceeds 15%, the observer pauses the LP and routes the cascade to a human moderator.

2.3 Data and polarisation discovery

2.3.1 Datasets

WICO Graph. Twitter cascades organised into three folders: `5G_Conspiracy_Graphs` and `Other_Graphs` for misinformation, `Non_Conspiracy_Graphs` for verified content. Each cascade folder contains an `edges.txt` file and a `nodes.csv` file. We use WICO Graph for SBM fitting and for the end-to-end cascade-following experiments.

Twitter15 and Twitter16. Per-tweet propagation trees with four-class rumour labels: `true`, `false`, `unverified`, `non-rumor`. We use them for BiGCN training and evaluation only.

2.3.2 Edge orientation

WICO Graph stores its edges in the inverse direction relative to content flow: the file encodes “this user retweeted that user”, which gives an arrow pointing from the retweeter to the original poster, while content actually moves the other way. We correct this at load time by calling `G.reverse(copy=True)` on every cascade graph as it enters the pipeline. After reversal, edge $u \rightarrow v$ means user u shared content that reached user v . Without this correction the cascade root identification picks a leaf instead of the author, the BFS walks backwards in time, and the SBM fitter counts transfers in the wrong direction.

2.3.3 Polarisation discovery: argued choices

The polarisation partition $\mathcal{C}$ is recovered from the union of all observed cascades by Louvain modularity clustering followed by a small-class merging step. Two configuration parameters control the procedure: Louvain’s resolution and the minimum-class-fraction floor below which a community is merged into its largest neighbour. We use resolution 0.5 and floor 0.05. Both values are deliberate.

Resolution. At resolution 0.1, Louvain returns three large balanced communities and the 5G-conspiracy subgraph is absorbed into a larger neighbour. This loses the cross-class pathway that drives the entire LP signal: when the misinformation community lives inside a larger generic community, the

Table 1: Fitted SBM transfer matrices on WICO Graph after orientation correction and 4-class merging. The dominant cross-class pathway in b^- ($C_3 \leftrightarrow C_4$) is shown in bold. All other off-diagonal entries are at or near the Laplace floor in both matrices.

$u \setminus v$	C_1	C_2	C_3	C_4
b^+ (<i>verified content</i>)				
C_1	1.0×10^{-4}	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$
C_2	$< 10^{-6}$	1.0×10^{-4}	$< 10^{-6}$	$< 10^{-6}$
C_3	$< 10^{-6}$	$< 10^{-6}$	7.0×10^{-4}	$< 10^{-6}$
C_4	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$	3.0×10^{-4}
b^- (<i>misinformation</i>)				
C_1	1.0×10^{-4}	$< 10^{-6}$	$< 10^{-6}$	$< 10^{-6}$
C_2	$< 10^{-6}$	1.0×10^{-4}	$< 10^{-6}$	$< 10^{-6}$
C_3	$< 10^{-6}$	$< 10^{-6}$	6.0×10^{-4}	1.00
C_4	$< 10^{-6}$	$< 10^{-6}$	0.93	4.0×10^{-4}

SBM’s off-diagonal asymmetry between b^+ and b^- collapses. At resolution 0.5, the conspiracy subgraph emerges as a distinct community.

Merging floor. The conspiracy class contains 8,484 nodes, which is 7.43% of the network. A 10% floor would merge it away even after Louvain has identified it correctly. A 5% floor preserves it.

These two values together — resolution 0.5 and merging floor 0.05 — are what make the suppression layer functional on this dataset. Neither follows automatically from the original methodology; both required iteration. The final partition has $k = 4$ classes with sizes [38113, 54956, 8484, 12682] across 114,235 nodes.

2.3.4 Fitted matrices

We estimate b^+ and b^- by frequentist counting on labelled cascades, normalised per pair of (sharing user, target user). Cells with fewer than five observations are blended toward their row mean to avoid extreme values from sparse data, and a Laplace floor of 10^{-8} is applied. Table 1 reports both matrices.

Three observations follow. Both matrices are diagonal-dominant — content of either type stays predominantly within its source polarisation class. Every off-diagonal cell of b^+ is at or near the floor: verified content does not cross polarisation classes in this dataset. b^- has one large cross-class pathway, $C_3 \leftrightarrow C_4$, where b^- reaches the unit cap (clipped from a raw estimate above 1.0, reflecting a heavily connected sharer-target subgraph in the conspiracy cascades) and the corresponding b^+ entries are at

the floor. The LP’s entire suppression action concentrates on these two off-diagonal entries of d^* . This is what the polarisation-discovery configuration of §2.3 buys us: a single resolved class pair on which the suppression mechanism has actionable signal.

2.4 Cascade-following implementation

The simulator walks an observed WICO cascade in breadth-first order from its root, calling the LP optimiser at every breadth level and applying the resulting dropout matrix as a Bernoulli mask on the forward edges. The function signature is

```
def simulate_cascade_following(
    G: nx.DiGraph, partition: dict, root: str, sbm: SBM,
    alpha: Optional[float], lam: Optional[float],
    global_class_sizes: np.ndarray, seed: int = 42,
) -> tuple[int, list[str]]
```

and the function is deterministic given its arguments. Three implementation choices warrant explicit description.

Global susceptibility counts. The susceptible-class vector at every step is computed against the global polarisation class sizes — the full 114,235-node network — rather than against the cascade-local subgraph. The LP was derived for the full-network process, and a cascade-local susceptible vector would underestimate the constraint denominator and force the LP to under-suppress.

Single-seed normalisation at $t = 0$. WICO records each cascade after the original tweet has already been published, so the recorded depth-1 retweeters are observed cascade members rather than candidate transfers. The simulator initialises the reachable set with the root and its depth-1 successors. The LP at step 0 uses a normalised infected vector with a single unit mass in the root’s class rather than the actual depth-1 count. Without this normalisation the constraint $\alpha|I_0|$ scales with whatever the depth-1 fan-out happens to be, and the LP over-preserved both content types when the original tweet had many initial retweeters.

Soft-LP fallback handling. When the feasibility condition fails — typically late in a cascade, when $|I_t|$ has grown but the susceptible classes are largely depleted — the simulator switches to the soft LP and continues. The per-step LP-type log records which solver branch fired at each step. This is the diagnostic that lets us tell whether a cascade was suppressed by the constrained LP or by the soft fallback, and it is the input to the (specified, not implemented) bias-audit module.

2.5 BiGCN classifier and integration

The BiGCN component is a Bidirectional Graph Convolutional Network classifier on cascade trees. It is the only learned component in InfoGuard.

2.5.1 Architecture

The classifier has two parallel GCN branches and a small classifier head. The first branch reads the cascade tree top-down (root toward leaves); the second reads it bottom-up (leaves toward root). Each branch is two `GCNConv` layers with hidden dimension 256 and output dimension 128. Branch outputs are mean-pooled per graph, concatenated to a 256-dimensional graph embedding, and passed through a two-layer feedforward classifier head ($256 \rightarrow 128 \rightarrow 4$) with ReLU activations and dropout $p = 0.5$ between layers. The output is a four-way logit over the Twitter15 / Twitter16 label set.

2.5.2 Input feature design

Each node carries 772 input features: a 768-dimensional RoBERTa-base CLS-token embedding of the source tweet, broadcast unchanged to every node in the tree, plus four per-node structural features.

The broadcast. Per-node tweet text is not distributed for either Twitter15 or Twitter16 under Twitter Terms of Service. The dataset provides only the source-tweet text. We make the source embedding visible at every node by broadcasting it; every reply node carries the same 768-dimensional vector. This is semantically defensible — every reply exists in the context of the source claim — and it is empirically necessary. Replacing the broadcast embedding with zero vectors at non-root nodes drops 4-class accuracy from above 0.85 to roughly 0.68 on the same architecture in our internal ablation. The structural features are insufficient on their own to recover the source-claim context lost beyond two GCN hops.

The four structural features. Each node also carries a binary root indicator, normalised BFS depth (depth divided by maximum depth in the tree), normalised out-degree, and normalised descendant count. These let the network distinguish root from replies and recover the propagation-tree shape that the broadcast erases from the text component.

2.5.3 Training

We train each fold for up to 200 epochs with the Adam optimiser, learning rate 5×10^{-4} , weight decay 5×10^{-4} , and batch size 32. The loss is cross-entropy over the four-class objective with label smoothing 0.05. Dropout in the GCN branches and the classifier head is 0.5; gradient clipping is 5.0.

The checkpoint per fold is saved on best validation macro-F1, not best validation loss. With label smoothing 0.05, the cross-entropy loss has a non-zero floor and tracks classification quality only loosely; selecting on F1 yields the best-classifying checkpoint at the cost of a slightly noisier early-stopping signal. Early stopping monitors validation F1 with patience 20 epochs.

We use stratified 5-fold cross-validation. Twitter15 has approximately 1,490 labelled source tweets; 5-fold leaves about 298 trees in each held-out fold, large enough for per-class F1 to be stable on the rarest of the four classes. Halving the per-fold test count by moving to 10-fold pushes fold-to-fold F1 variance above the differences we are trying to measure between training configurations. Each fold runs up to 200 epochs and 5-fold completes within the available single-GPU budget.

2.5.4 Predictor interface

The `Predictor` class exposes a single inference method

```
predictor.predict_from_digraph(G, tweet_id, tweet_text) -> result
```

returning a result object with three fields used downstream: `binary_label` $\in \{\text{true}, \text{false}, \text{uncertain}\}$, `confidence` (the maximum softmax probability over the four-class output), and `high_confidence` (a boolean equal to `confidence` ≥ 0.65).

The threshold 0.65 is the only point at which the classifier touches the rest of the pipeline. We chose this value to keep the high-confidence subset small enough that classification mistakes within it are rare while large enough that the SBM fitter has enough labelled cascades to refit with. Sweeping the threshold is a future-work item; the current value is the operating point at which fold-0 classifier behaviour on the validation set is tolerable for an input filter.

2.5.5 Integration logic

When `high_confidence` is true, the predicted label is forwarded to the SBM fitter via `add_cascade(G,`

Table 2: Cascade-size discrimination on WICO Graph. Each cell averages 500 cascades sampled with replacement from the WICO pool restricted to cascades with at least 20 downstream nodes.

(α, λ)	$\mathbb{E}[R_\infty]$		$\Pr[R_\infty < 5]$	
	verified	misinfo.	verified	misinfo.
control	56.1	59.8	0.00	0.00
(1.5, 1.0)	42.8	38.5	0.05	0.14
(2.0, 1.5)	45.0	41.8	0.04	0.10
(3.0, 2.0)	48.8	46.2	0.02	0.08

label, confidence). The cascade enters the next refit of b^+ and b^- at full weight. When **high_confidence** is false, the cascade is dropped from the fitter input.

Either way, the classifier produces nothing the LP can read directly. A wrong BiGCN prediction can shift the fitted matrices over time but cannot block any single transfer in any cascade. This is the architectural property that lets us run a classifier with binary accuracy 0.84 inside a system whose suppression layer must hold itself to a much stricter standard: the classifier is a slow-acting input filter, not a fast-acting moderator. The only error mode classifier mistakes can produce is matrix drift over many refit cycles, and that effect is bounded by the refit cadence and how the matrix update blends old and new estimates.

2.6 Experimental setup and results

2.6.1 Setup

We run end-to-end cascade-following on WICO Graph at four (α, λ) settings:

$$(\alpha, \lambda) \in \{\text{control}, (1.5, 1.0), (2.0, 1.5), (3.0, 2.0)\}.$$

The control row uses no dropout: $d \equiv 1$ at every step. We sample 500 cascades per setting per label, independently with replacement, from the WICO pool restricted to cascades whose root reaches at least 20 downstream nodes. For each cell we record R_∞ and report two summary statistics per label: the mean cascade size $\mathbb{E}[R_\infty]$ and the early-collapse probability $\Pr[R_\infty < 5]$.

2.6.2 Cascade-size discrimination

The control row gives the unaltered cascade dynamics on WICO: misinformation cascades reach 59.8 users on average against 56.1 for verified content. Both

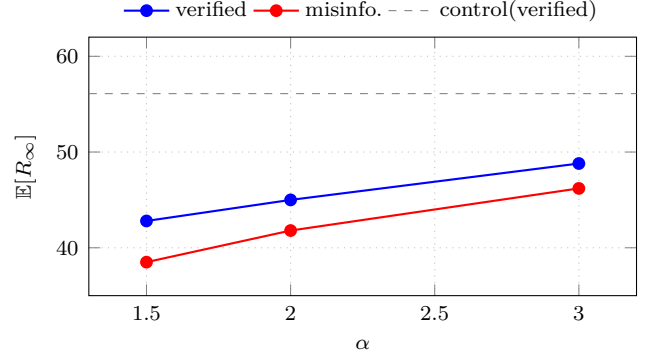


Figure 2: Mean cascade size $\mathbb{E}[R_\infty]$ on WICO as α varies, with paired $\lambda \in \{1.0, 1.5, 2.0\}$. The dashed line marks the verified-content control level. Misinformation sits below verified at every measured α , and the gap closes as α tightens.

content types collapse early ($R_\infty < 5$) on a negligible fraction of cascades.

At $(\alpha, \lambda) = (1.5, 1.0)$ the cascade-size ordering reverses. Verified content drops from 56.1 to 42.8, a 24% reduction. Misinformation drops from 59.8 to 38.5, a 36% reduction. The early-collapse probability rises to 0.05 for verified content and 0.14 for misinformation, a $2.8\times$ ratio against misinformation. As α tightens, the LP keeps more transfers active across the board: at $(2.0, 1.5)$ both means recover (45.0/41.8) and at $(3.0, 2.0)$ they recover further (48.8/46.2). The verified-vs-misinformation ordering remains correct at every measured setting.

2.6.3 Discrimination across the α sweep

Figure 2 plots $\mathbb{E}[R_\infty]$ for both content types as a function of α . The vertical gap between the two curves is the discrimination available at that setting; it is largest at $\alpha = 1.5$ and narrows monotonically as α grows.

2.6.4 BiGCN classifier performance

Per-class F1 on the 4-class objective at fold 0 is 0.74 for **true**, 0.66 for **false**, 0.58 for **unverified**, and 0.65 for **non-rumor**. The dominant error source is the **unverified** class, which in Twitter15 marks rumours that had not been confirmed true or false at labelling time and is therefore defined by the labelling process rather than by any property of the content itself.

Table 3: BiGCN performance on Twitter15 under 5-fold cross-validation. Fold 0 reported; folds 1–4 pending. Binary metrics collapse **non-rumor** into **true** and exclude **unverified**.

Fold	4-class Acc	4-class F1	Binary Acc	Binary F1
0	0.66	0.66	0.84	0.76
1	—	—	—	—
2	—	—	—	—
3	—	—	—	—
4	—	—	—	—
Mean [†]	0.66	0.66	0.84	0.76

[†] Computed over fold 0 only at this draft.

2.6.5 Components not currently producible

Two of the planned outputs are deferred. The hybrid SBM-only versus SBM-plus-BiGCN comparison requires per-cascade text on WICO, which is not distributed; on WICO the BiGCN handoff into the SBM fitter currently falls back to folder labels. The bias-audit output is not producible because the audit module is specified but not implemented.

2.7 Use-case walkthrough

The deployment target is a wartime or disaster information environment, with the Azerbaijani context as the explicit example. The walkthrough below describes what the system would do during a single live cascade. It is illustrative; the Azerbaijani-language pipeline has not been built.

Step 1: Cascade begins. A source tweet is published during an active emergency. The platform observes the first retweets within seconds. The cascade graph G accumulates in real time.

Step 2: BiGCN classification. The classifier runs once on the source tweet and the partial cascade. It returns `binary_label = false` with confidence 0.81. Because the confidence exceeds the gating threshold, the cascade is forwarded to the SBM fitter. The classification does not act on the current cascade.

Step 3: LP runs at every breadth level. At every step of the cascade-following loop the system computes per-class susceptible and infected counts, evaluates the feasibility condition, and solves the primary or soft LP. The dropout matrix d^* is applied as a Bernoulli mask on every forward edge.

Step 4: Bias audit (when implemented). The audit observer reads d^* at every step and accumulates per-content-category dropout rates. If the largest pairwise gap between categories exceeds 15%, the observer halts the LP loop and routes the cascade to a human moderator.

Operational ethic.

InfoShield is designed for deployment against genuine misinformation in emergency scenarios, including public-health crises, disaster response, and foreign disinformation campaigns. The bias audit module is a mandatory component that must run continuously during any deployment. Differential dropout rates exceeding 15% between political content categories automatically trigger human review and operational pause. This system must never be deployed against legitimate political speech. The distinction between misinformation and political opposition is an editorial judgement that cannot be delegated to an algorithm; human oversight is non-negotiable.

3 Results and Discussion

The measurements support a single claim: the network-level intervention reduces misinformation cascades more than verified cascades on WICO, and it does so without reading post content at suppression time. The strongest single piece of evidence is the sign flip in mean cascade size between control and $(\alpha, \lambda) = (1.5, 1.0)$. At control, misinformation cascades reach more users than verified cascades. Under the intervention, the ordering reverses.

The mechanism is visible in the fitted matrices. Verified content stays within polarisation class — every off-diagonal entry of b^+ is at the Laplace floor. Misinformation crosses one specific class boundary: $b^-[3, 4] = 1.00$ and $b^-[4, 3] = 0.93$, with the corresponding b^+ entries at the floor. The LP responds by setting $d^*[3, 4]$ and $d^*[4, 3]$ low, which suppresses the dominant misinformation transfer pathway while leaving the diagonal transfer rates that carry verified content largely intact. The polarisation-discovery configuration of §2.3 is what enables this: at lower Louvain resolution or a higher merging floor, the conspiracy class would not be resolved separately and the $C_3 \leftrightarrow C_4$ signal would not exist.

Two limitations bound the strength of the conclusion. The end-to-end measurements use folder labels rather than BiGCN labels, because WICO Graph does not distribute per-cascade text. The bias-audit module is specified but not implemented, so the discrimination guarantee is unenforced in the current pipeline. The present results demonstrate technical feasibility of network-level discrimination on this dataset under these polarisation-discovery and operating-parameter choices. They do not demonstrate deployment readiness.

4 Conclusion

InfoGuard suppresses misinformation cascades without classifying any individual post. The system fits two SBM transfer matrices, b^+ for verified content and b^- for misinformation, and finds that the structural asymmetry concentrates in a single class pair, $C_3 \leftrightarrow C_4$ ($b^- = 1.00$ clipped and 0.93; the corresponding b^+ entries at the Laplace floor). The system exploits this asymmetry through a per-step linear program over a $k \times k$ dropout matrix indexed only by polarisation class — never by post text. At $(\alpha, \lambda) = (1.5, 1.0)$ the mean misinformation cascade size on WICO falls from 59.8 to 38.5; the mean verified-content cascade size falls from 56.1 to 42.8. The cascade-size ordering reverses under the intervention and remains correct at all three measured settings.

The contributions are the four pieces of engineering that turn the optimisation framework of Bayiz and Topcu (2022) into a deployable system: a confidence-gated BiGCN classifier as input filter to the SBM fitter, an argued polarisation-discovery configuration without which the LP signal disappears, an emergency-deployment framing with a specified bias-audit module, and a future-work direction for retrieval-augmented counter-content.

Weaknesses

Bias audit not implemented. The bias-audit module is the component that decides whether the system is acting against misinformation or against political opposition. Without it, the discrimination guarantee is unenforceable in practice. The 15% threshold and the human-review trip-wire exist on paper, not in code. Until the audit is implemented and validated, the system cannot be deployed.

Classifier error drifts the fitter over time. BiGCN binary accuracy is 0.84. The remaining 16% of high-confidence predictions are wrong and enter the SBM fitter with the wrong label. Over many refit cycles, drift can pull b^+ and b^- away from the true asymmetry that the LP relies on. The current pipeline has no drift detector.

Single resolved cross-class pathway. The discrimination signal on WICO concentrates in one class pair. A dataset whose misinformation does not produce such a sharply localised pathway — broader cross-class spread, multiple pathways, or pathways that overlap with verified-content transfer routes — would offer the LP much weaker discrimination. Whether the WICO pattern generalises to other emergency-context datasets is an open question.

Coarse polarisation classes. The pipeline reduces the user base to $k = 4$ classes. Real polarisation is continuous and shifts over time. A four-class partition discretises a finer property and freezes it at fitting time. Periodic refitting would help; refitting without restarting the running LP is an open implementation question.

Single-language evaluation. All training and evaluation use English-language data. The Azerbaijani deployment scenario is illustrative; nothing in the measured pipeline demonstrates that the methodology transfers to Azerbaijani. The text encoder, the propagation patterns, and the polarisation structure all need to be re-fitted on Azerbaijani-language cascade data before any operational claim is made.

Future work

Counter-content generation via retrieval-augmented language models. The system as built is purely defensive: it stalls misinformation cascades but produces no replacement information. The next extension is an offensive component. We propose a Retrieval-Augmented Generation (RAG) module that consumes a continuously updated index of trusted sources — official government communications, verified news outlets, primary scientific publications, emergency-response bulletins — and generates verified counter-narratives grounded in retrieved passages from that index. When the LP suppresses a misinformation cascade on a given topic, the RAG module generates a counter-narrative on the same

topic and seeds it into the polarisation classes the misinformation was reaching. Three properties of the design matter for the safety story. First, the trusted-source index is maintained by editorial staff, not by the model; the RAG module’s role is generation under retrieval constraint, not source curation. Second, generation is constrained to cite its retrieval sources in the output, which gives downstream readers an audit trail and reduces hallucination risk. Third, the index updates dynamically — new bulletins, new scientific results, new official statements enter the retrieval pool as they are published — so the counter-narrative reflects the current state of trusted information rather than a frozen snapshot. The same bias-audit constraint that gates the LP must gate the generator: a counter-narrative whose dispersal crosses category boundaries by more than 15% triggers the same human-review pause as a dropout matrix that does so.

Implementing the bias audit. The audit module is the highest-priority engineering task. The specified behaviour is implementable as a side observer on the LP output and does not require changes to the core pipeline. The same observer must be extended to read the RAG generator’s outputs once the counter-content layer is in place.

Adaptive α and λ . Both safety parameters are currently fixed per run. A controller that adjusts them per cascade based on observed early dynamics would close the loop and remove the manual sweep. The reward signal is the cascade-discrimination metric reported in §2.6.

Azerbaijani-language extension. Porting to Azerbaijani requires three independent components: an Azerbaijani text encoder (replacing RoBERTa-base), Azerbaijani-language labelled cascade data for fitting b^+ and b^- , and an Azerbaijani-language trusted-source retrieval store for the RAG module. The mathematical core does not change. The data and language assets do.

References

- Bayiz, Y. E., & Topcu, U. (2022). *Countering misinformation on social networks using graph alterations* (arXiv:2211.04617). arXiv.
- Bian, T., Xiao, X., Xu, T., Zhao, P., Huang, W., Rong, Y., & Huang, J. (2020). Rumor detection on social media with bi-directional graph convolutional networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(1), 549–556.
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008.
- Del Vicario, M., Bessi, A., Zollo, F., Petroni, F., Scala, A., Caldarelli, G., Stanley, H. E., & Quattrociocchi, W. (2016). The spreading of misinformation online. *Proceedings of the National Academy of Sciences*, 113(3), 554–559.
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).
- Holland, P. W., Laskey, K. B., & Leinhardt, S. (1983). Stochastic blockmodels: First steps. *Social Networks*, 5(2), 109–137.
- Huangfu, Q., & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119–142.
- Kempe, D., Kleinberg, J., & Tardos, É. (2003). Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 137–146). Association for Computing Machinery.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*.
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *5th International Conference on Learning Representations*.

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach* (arXiv:1907.11692). arXiv.
- Ma, J., Gao, W., Mitra, P., Kwon, S., Jansen, B. J., Wong, K.-F., & Cha, M. (2016). Detecting rumors from microblogs with recurrent neural networks. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence* (pp. 3818–3824).
- Ma, J., Gao, W., & Wong, K.-F. (2017). Detect rumors in microblog posts using propagation structure via kernel learning. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics* (Vol. 1, pp. 708–717). Association for Computational Linguistics.
- Newman, M. E. J. (2002). Spread of epidemic disease on networks. *Physical Review E*, 66(1), 016128.
- Pennycook, G., & Rand, D. G. (2021). The psychology of fake news. *Trends in Cognitive Sciences*, 25(5), 388–402.
- Pogorelov, K., Schroeder, D. T., Filkuková, P., Brenner, S., & Langguth, J. (2021). WICO Text: A labeled dataset of conspiracy theory and 5G-corona misinformation tweets. In *Proceedings of the 2021 Workshop on Open Challenges in Online Social Networks* (pp. 21–25). Association for Computing Machinery.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... Vázquez-Baeza, Y. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272.
- Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. *Science*, 359(6380), 1146–1151.